CCAR-P STUDY PACKAGE · CLAUDE CERTIFIED ARCHITECT – PROFESSIONAL

Overview & Big Picture / 全局图

Exam Guide v1.0 · 63 items · 120 min · pass = 720/1000 · Peter's exam: Wed Dec 2, 2026, 10:00am

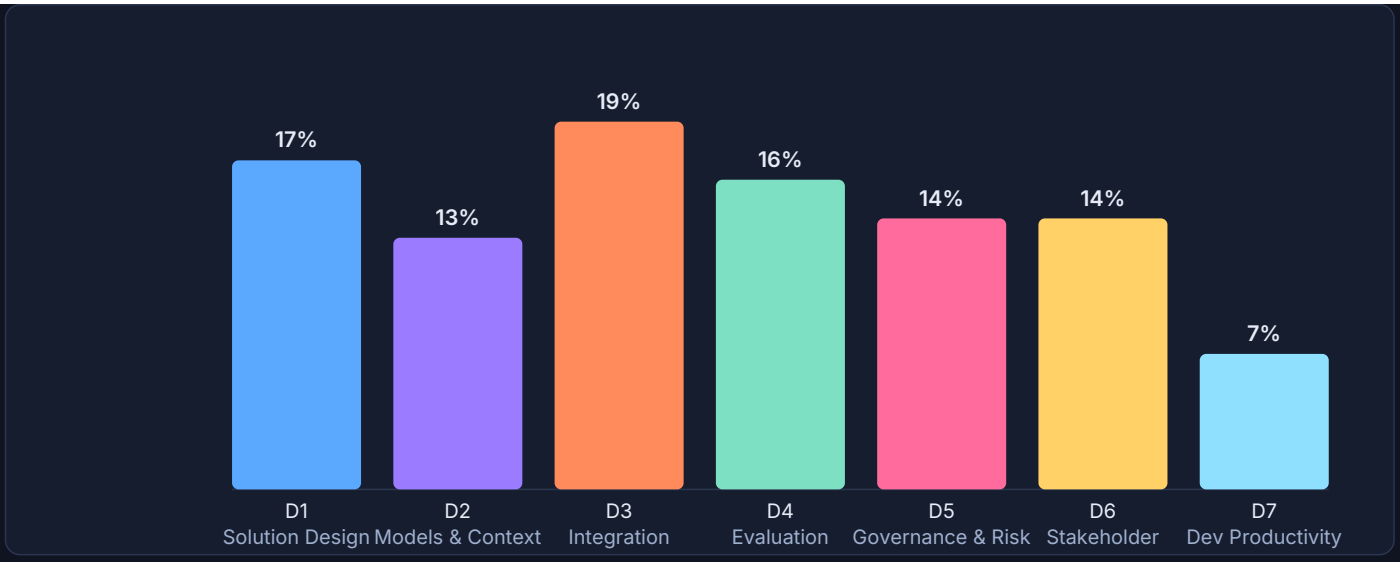
1. How this package maps to the exam read first

Five Skilljar modules feed seven exam domains. Domain weight is not module weight — D3 Integration (19%, the single biggest domain) is split across Module 1 (RAG, protocols, context strategy) and Module 2 (auth, observability, accuracy/latency). Treat the table below as the map every other page in this package points back to.

Module (course)	Exam domain(s)	Weight	Teaches you to
M1 Claude Platform & Solution Design	D1, D2, part of D3 (RAG, protocols, progressive vs. monolithic)	17% + 13% + part of 19%	Turn a fuzzy business problem into a defensible architecture
M2 Enterprise Integration & Production	D3, D4 (plus D1 sizing/ROI)	19% + 16%	Push a POC to production: eval, cost, reliability, integration, experiments
M3 Responsible AI, Safety & Risk	D5 (plus D3 security gaps)	14%	Build the safety stack: where each control sits, how it fails
M4 Stakeholder Engagement & Lifecycle	D6 (plus D3/D5 routing)	14%	Run discovery, frame tradeoffs, close feedback loops, document, prove outcomes
M5 Team Enablement & Ops Productivity	D7 (plus D4 diagnosis)	7%	Hand off a live system so it runs without you

中文 五门课程对应七个考试领域。M1 和 M2 合计约占考试的 65%。D3 集成（19%，最大单一领域）分布在 M1 和 M2 中。

2. Blueprint weights D1–D7

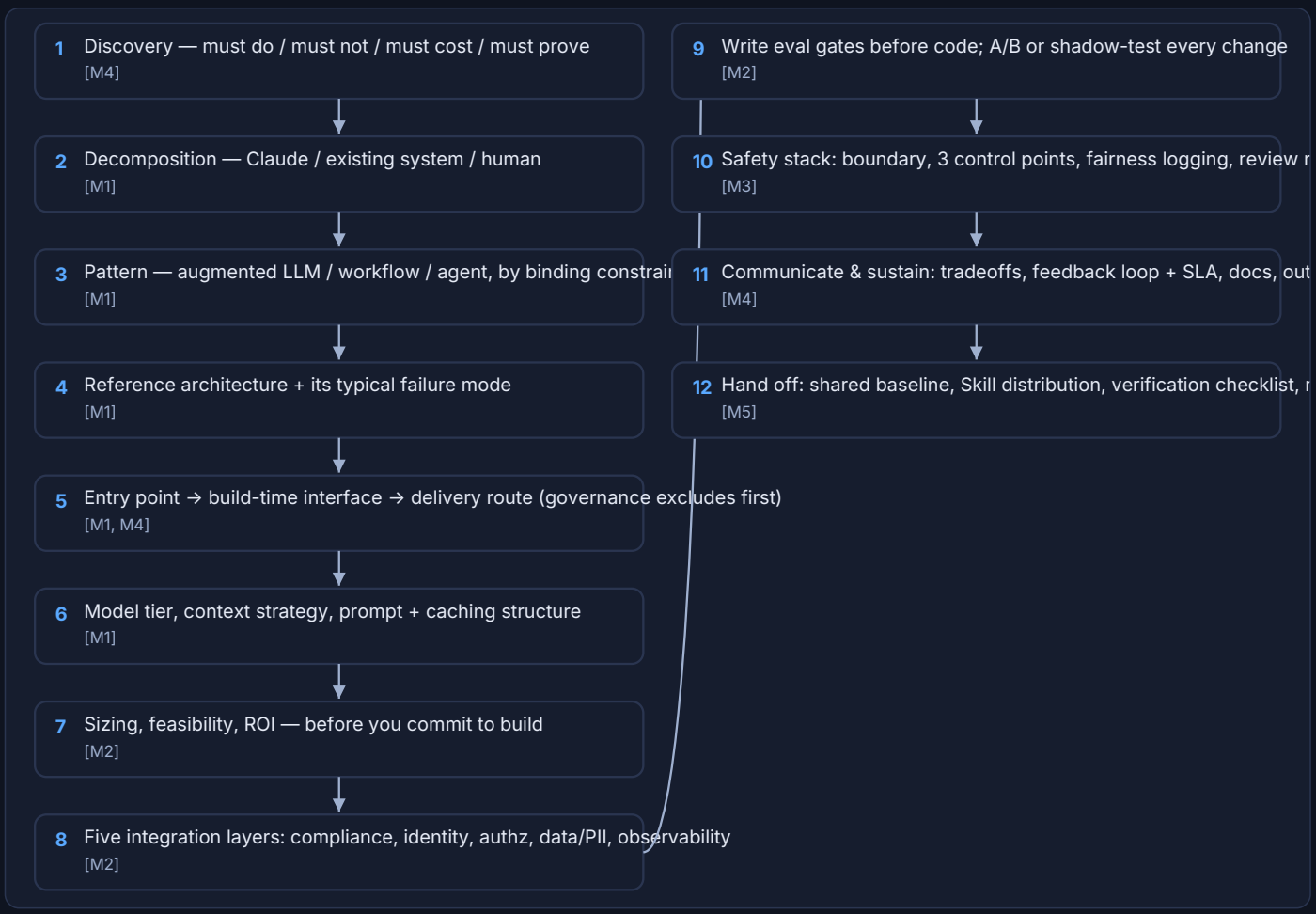


Scored-item share by content domain (Exam Guide §6). D3 Integration is the single largest domain.

3. The architect's decision chain

one spine for every scenario

Every scenario item lands on one step of this chain. Name the step before you touch the options — it tells you which rule set applies.



12-step architect decision chain. Every exam scenario lands on one box — identify it before reading the options.

4. Ten judgment rules tie-breakers

When two options both look right, one of these usually settles it.

#	Rule	Where
1	**Remove > prevent > instruct > detect.** Remove the capability if you can; else a deterministic preventive control (hook, schema, DB permission, authz); only then instruction; logging/review last. (Enforcement ladder — Sample Q1.)	M1,M2,M3
2	**Solve the risk the stem names.** Distractors solve a real but different risk.	all
3	**Lightest solution that solves the problem.** Fewest primitives, zero-shot first, workflow before agent, default Sonnet, MCP only when reused.	M1
4	**Governance first.** Compliance, procurement, data residency exclude options before cost, latency, convenience.	M1,M2,M4
5	**Rules/real-time facts/computation → code or systems. Language work → Claude. High-stakes decisions → a human.**	M1
6	**Stable knowledge → retrieval. Live state → tools.** Using RAG instead of a tool call is the most common mistake.	M1
7	**Every change is an eval-gated release**, with the threshold set before the run (model swap, prompt edit, context change).	M1,M2
8	**Look at distributions and segments, not averages.** Token distribution, p95 latency, eval score by segment.	M2
9	**Structure beats sentences.** Output contract, schema, hook, authorization > a rule written into the prompt.	M1,M3
10	**Prevention beats detection.** Ask: does this stop the bad output, or only help me find it afterward?	M1,M3

Common distractor patterns / 常见干扰项模式

- More context always improves quality.
- The largest model is always safest.
- A prompt instruction, a CLAUDE.md line, or a Skill is an enforceable guardrail.
- Logging or a confirmation prompt equals least privilege.
- MCP is always better than calling the API directly.
- When multi-agent breaks, rerun the whole pipeline or add more logging.
- Average latency or average accuracy met the SLA.
- The vendor already ran a bias evaluation, so fairness is handled.
- When the stem signals latency or a configuration problem, "none required" is the safe answer.

5. Four model properties foundation of everything else

Property	Good at	Bad at	Design response
Next-token prediction / non-determinism	Pattern-rich language work: summarize, rewrite, explain, draft	Exact details: names, dates, citations, statistics; same input, different output	**Eval.** One clean demo is not evidence. Verification loops, citations, facts go to tools
Knowledge boundary	Common, stable, well-documented topics	Rare, private, recent, fast-changing facts — stated confidently anyway	**Retrieval and tools.** External systems are the source of truth
Working memory (finite context)	Anything inside the context window	The window edge is a cliff; quality degrades before the hard limit	**Context strategy** is a design decision: progressive loading, chunking, compaction
Steerability / confidence ≠ correctness	Short, specific, verifiable, formatted instructions	Vague goals, long reasoning chains, precise arithmetic — wrong answers delivered just as confidently	**HITL placement and verification.** Schema, code execution, review where errors cost most

Mnemonic: Unpredictable → test it. Doesn't know → go retrieve it. Doesn't fit → make a choice. Can't tell right from wrong → go verify.

中文 不可预测 → 测试它。不知道 → 去取。装不下 → 做选择。分不清对错 → 去核查。判定规则：针对根本原因，而不是症状。

6. AI Fluency 4D framework 新增 / added

Not a formal Section 6 objective, but stems borrow its vocabulary.

Capability	Means	Maps to architecture work	Domain
Delegation	Decide what goes to AI, human, or both. Modes: automation, augmentation, agency	Decomposition — who owns each capability	D1
Description	Tell the AI the product, process, and performance you want	Prompt & context engineering; stakeholder framing	D2, D6
Discernment	Judge output, reasoning, behavior: accept / revise / reject	Eval, LLM-as-judge, HITL placement	D4, D5
Diligence	Own responsibility: use tools responsibly, stay transparent, verify before deploy	Governance, checklists, accountability checkpoints	D5, D6, D7

中文 描述和辨别构成一个循环：描述、评判、改进。

7. Exam mechanics at a glance

Format

- 63 items, 120 minutes (≈1.9 min/item)
- Multiple-choice and multiple-response (stem states how many to select)
- Scaled score 100–1000, pass = 720, criterion-referenced
- Score report: pass/fail + domain percent-correct (domain % does not decide pass/fail)

Policy need-to-knows

- Retakes: 14 / 30 / 90-day waits after 1st/2nd/3rd fail; max 4 attempts / 12 months
- Cancel/reschedule ≥24h ahead or forfeit the $175 fee
- Credential valid 12 months; free renewal assessment on time
- No API parameter names, CLI flags, or file paths — Professional reads like a client mission, not a syntax quiz

---

8. Your logged weak spots across all modules

- D2 Select appropriate Claude models based on trade-offs
- D2 Optimize context windows and manage token usage
- D3 Evaluate connection protocols (MCP, API/CLI, agent-to-agent)
- D3 Evaluate tool/agent configuration for capability bloat
- D3 Evaluate progressive discovery vs. monolithic context strategy
- D3 Analyze observability challenges and select monitoring strategies at scale
- D4 Design evaluation datasets and test frameworks using mixed methodologies
- D4 Evaluate accuracy-latency trade-offs and justify configuration decisions
- D4 Conduct A/B testing and iterative improvements
- D5 Implement guardrails and safety controls
- D6 Document architectures and provide implementation guidance
- D6 Manage stakeholder feedback loops and expectation alignment

Drill these first every session. Spaced repetition: miss → stage 0 (review next session) → pass → stage 1 (+3 days) → pass → stage 2 (+7 days) → pass → cleared. Any miss resets to stage 0.

Module 1: Platform & Solution Design →

MODULE 1 · 34 PAGES / 11 CHECKPOINTS IN COURSE

Claude Platform & Solution Design

Exam domains: D1 (most), D2 (models, prompting, caching, Skills), D3 (RAG, protocols, progressive vs. monolithic), D5 (regulated-industry routing)

This module answers: facing a business problem, what does Claude own? What shape does the work take? Which reference architecture fits? Where does the solution touch Claude?

1.1 Four decisions, in order

1. **Decomposition** — what Claude owns, what the existing system owns, what a human owns.
2. **Pattern** — augmented call, workflow, or agent.
3. **Reference architecture** — which mature shape applies.
4. **Touchpoints** — entry point, model, context strategy, delivery route.

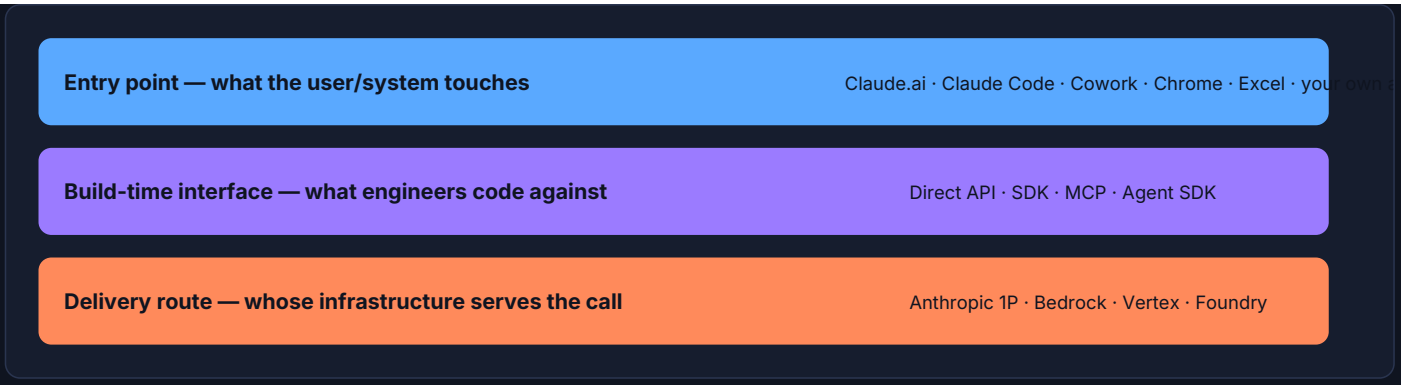
Getting the order wrong causes expensive rework. The habit the course wants: say out loud which decision you're facing right now, because each one rewards a different move.

1.2 How Claude behaves foundation

Context overflow behavior (verify exact codes before stating): a prompt too long is rejected before generation (too many tokens → 400 `invalid_request_error`; payload too large → 413 `request_too_large`). If the prompt fits but output hits the cap, generation stops early with a context-window-class stop reason. Measure with the response's `usage` field and the token-count API — never estimate from character ratios.

Course case: a reconciliation pipeline shipped with zero checks because a demo looked deterministic. Non-determinism is the entire reason an eval framework exists.

1.3 Three independent platform layers



Every deployment has all three layers. Conflating them is how a developer tool ends up in a non-developer's hands.

1.4 Seven primitives, one job each

Primitive	Sole job	Use when
Tools	Act	Model needs to call a function: fetch a result or perform an action
MCP	Connect	The same tool set must serve multiple Claude clients (reuse)
Subagents	Isolate / parallelize	A scoped subtask needs a clean independent context, or needs to run in parallel
Hooks	Guarantee	A rule must hold every time — deterministic code on lifecycle events the model cannot skip
Skills	Package a procedure	A repeatable process needs versioning, sharing, governance (SKILL.md + optional scripts)
Agent Teams	Coordinate peers	Several agents each own part of the goal
Dynamic Workflows	Compose at runtime	Steps must assemble at runtime; they can't be fixed in advance

Rule: use the fewest primitives the requirement needs. Every added one adds latency, tokens, and a surface that needs observability and governance. Patterns (augmented call, workflow, agent, multi-agent) are assemblies of these primitives.

Tool vs. MCP vs. Skill vs. Hook — four dimensions, not four alternatives

Dimension	Answers	Finance analogy
Tool	What the model can do (capability; model decides whether to call it)	A transaction code
MCP	How tools/resources/prompts are shared across clients (protocol)	A standard interface opening a transaction to multiple systems
Skill	How to do a procedure well (packaged expertise; can	An SOP or desk procedure

	specify which tools to use)	
Hook	What must happen (enforcement outside the model)	An automatic preventive edit check that blocks posting

A Skill sits on top of tools; tools are never built on top of a Skill. Packaging (Skill) and sharing (MCP) change distribution, not enforcement.

1.5 Decomposition and delegation D1

- **Claude:** language understanding, summarizing, drafting, planning, acting through tools, perception (reading a label photo).
- **Existing system:** anything already reliable — rules engine, system-of-record database, policy table, computation, email/payment systems.
- **Human:** judgment, exceptions, approvals, safety decisions, unrecoverable money.

Three ownership tests: reversibility, stakes, accountability. The output is a delegation map (AI owns / human owns / Claude drafts + human decides).

Reframe: don't ask "where can Claude help?" Ask "where do the four model properties make Claude more suitable than a system that already does this correctly?" Claude can attempt every step — that is not the test.

Practice: the three-question sort

- Is it a system-of-record fact, a calculation, or an eligibility rule? → **Existing system.** Claude can call it as a tool and narrate the result.
- Is it high-consequence, irreversible, over a threshold, or safety-related? → **Human.** Claude can draft or flag.
- Is it language or perception work with a human or system checking the next step? → **Claude.**
- Split compound steps: "tell the engineer whether warranty applies" = system decides, Claude narrates. "Issue the refund" = system determines eligibility, human approves payment.

Exam trap

Stuffing a deterministic business rule (threshold, eligibility, priority, coverage) into the prompt to "simplify" the design. It works cleanly in a demo and fails quietly on messy input, surfacing later through an audit rather than monitoring. Rules stay in code; Claude extracts, the system decides.

1.6 Pattern selection D1

Pattern	Control flow lives in	Use when
Augmented	One call, your code	Scoped, verifiable task; can add tools, retrieval, thinking

LLM	around it	
Workflow	Your code (named steps)	Steps are known in advance; error cost and observability matter
Agent	The model	The path genuinely can't be enumerated; errors are acceptable and recoverable

Workflow sub-patterns: chaining (sequential handoff), routing (classifier picks a path), parallelization (independent units run together, then merge or vote), evaluator-optimizer (generate, score, revise until it passes or hits a retry cap). Combine freely; start with the simplest.

Five deciding factors, in order — the first one that excludes a pattern wins: predictability, error cost, observability, latency budget, cost.

Production agents need: a lean tool set, a per-turn budget, explicit permissions, and stop conditions.

Quality-fix order: improve the prompt → add tools or retrieval → a stronger pattern (evaluator-optimizer) → fine-tuning last. Fine-tune only when volume is huge and inference cost is the binding constraint, or there's a hard latency ceiling, or a formatting-consistency problem the prompt can't solve — it locks you to a model version, and Claude fine-tuning access is limited and needs confirming with your account team.

Packaging spectrum: prompt-only → direct tool call → Skill. Pick Skill when the process repeats, crosses teams, and needs versioning and rollback.

Exam trap

"The task feels open-ended, so pick agent" — wrong if the real paths are enumerable. Agents create an audit gap: compliance can't find one deterministic approval step. An unpinned model version with no re-validation gate is a governance gap in any pattern. Agents are not necessarily more expensive — accumulated context and call count drive cost, not the pattern label.

1.7 Multi-agent orchestration D1

- **Orchestrator:** owns the goal, decomposition, delegation, aggregation. Does not do subtask work itself.
- **Subagents:** each owns one scoped unit, runs in a clean context, returns a structured result.
- **Design explicitly:** how you decompose, the result schema, conflict rules, gap rules.
- **Failure asymmetry:** a subagent failure is usually recoverable; an orchestrator failure usually isn't. Make subagent work idempotent and retryable (unit IDs; retries don't duplicate writes). Checkpoint orchestrator state.
- **Cost:** every subagent carries its own context, and the orchestrator pays again to aggregate. Use multi-agent only when the work truly exceeds one context.

Failure	Lands in	Design response
Subagent returns malformed or empty result	Subagent boundary (recoverable)	Validate every result; retry or reassign that unit; log the gap, don't fail the whole run
Two subagents conflict	Aggregation stage (recoverable)	Explicit conflict rule, or escalate to a human
Orchestrator loses goal or	Orchestrator (usually	Protect state; checkpoint so a failure can resume

state	unrecoverable)	
Fragmented trace	Observability (cuts across layers)	One shared trace ID across orchestrator and subagents

Practice: multi-agent design review checklist

- **Coverage check at aggregation:** returned result count must equal dispatched unit count; call out gaps explicitly ("9,850 of 10,000 classified, 150 missing"). The most dangerous failure is a confident, complete-looking summary built on missing units.
- Validate every subagent result; retry or flag failed units.
- Explicit conflict-resolution rule, or escalate to a human.
- One shared trace ID.
- HITL gate before anything irreversible; sample low-risk operations.
- Fix where the failure happened — don't paper over it upstream.

1.8 Reference architectures D1

Architecture	Done well	Typical failure
Agent	Goal + bounded tools + turn budget + success criteria	State-changing tools, unbounded autonomy, no human review
RAG	Stable corpus, chunked and indexed, relevant chunks placed in context	**Used for live state** (order status, inventory, queue, balance)
Document pipeline (evaluator-optimizer)	OCR → schema extraction → validation → exception routing	No exception path; low-confidence extractions with no human gate
Ticket triage (routing)	Detect intent → docs retrieval / transactional API / human approval	Retrieval used for live state; no human escalation; agent shipped before measuring a routed workflow
Coding agent	Agentic exploration, then deterministic edit/test/review steps	Edits and commits with no review gate; no language-specific regression eval

#1 mistake: letting retrieval do a tool call's job. Symptoms: stale chunks, answers that change on every reindex, answers that contradict the database. A better embedding model, faster refresh, or a tuned similarity threshold won't fix it. Call the system of record. Similar is not the same as true.

Only combine two architectures when their failure modes differ. Each needs its own eval set — a healthy overall score can hide one failing component.

1.9 RAG pipeline design D3

Two independent decisions. Chunking looks at the corpus itself (its structure). Indexing looks at how queries are phrased (the query mix the whole corpus faces).

Decision	Option	Use when
Chunking	Fixed-size + overlap	Homogeneous, weakly structured text; simplest
Chunking	Semantic	Each chunk must stand alone as coherent prose
Chunking	Hierarchical	Numbered-section documents (contracts, manuals, policy, FMR) where section context carries meaning — keep a section-path on each chunk
Indexing	Dense (embedding)	Paraphrase, concept, intent matching
Indexing	Sparse (BM25 / keyword)	Exact tokens: part numbers, citations, error codes, TAS/SFIS codes
Indexing	Hybrid + reciprocal rank fusion (RRF)	Mixed query patterns — the common production case. RRF sums 1/(k+rank) across lists; needs only rank, no score calibration; the lowest-tuning default

Always state the three-way tradeoff: retrieval quality vs. latency vs. maintenance cost. Smaller chunks + hybrid indexing raise quality and latency together. Larger chunks + dense-only is cheaper and faster but misses exact codes. An answer that skips the tradeoff is incomplete.

Risk: a confident answer built on the wrong chunk looks identical to a correct one. Measure retrieval with a labeled set; a pipeline that's correct at launch quietly degrades as documents grow.

Retrieval routing (live vs. static): a router can be keyword rules (cheap, brittle), a classifier step (testable), or agent-chosen (flexible, hardest to audit). Misrouting: a query that sounds static but needs live data gets sent to RAG and returns a stale answer; a query needing both gets single-label routed and half-answered. Fix: multi-label routing; default to the database with a data-as-of note when uncertain; log routing; score routing accuracy separately in the eval set.

1.10 Model and context strategy D2 D3

Four terms to keep separate: context window (what the model sees this call), retrieval (knowledge fetched and placed in the window), persistent application state (owned by your system, reached through tools), summaries/memory (managed by the application; the model has no native memory).

Model-tier rule: default Sonnet. Move up to Opus only when eval shows Sonnet falls short. Move down to Haiku only when eval shows the loss is acceptable. Not choosing a tier means you chose the most expensive one.

Per-step model selection (Haiku for classification, Sonnet for summarization, Opus only where proven necessary) is the standard fix for cost and latency overruns.

Context capacity planning: the window is a ceiling, not a target. Budget by the largest realistic conversation plus retrieval content, system prompt, scratch space, and growth margin.

Strategy	Fits	Failure point / cost

Monolithic	Scoped tasks, stable cacheable prefixes, reasoning that genuinely needs everything visible at once	Context accumulates per turn; cost grows linearly; quality drops before the hard limit; irrelevant entries cause wrong picks
Progressive (default)	Most production work: carry only what the next step needs; discover tools/docs on demand	Long-range coherence weakens; harder to cache; harder to reconstruct step-N inputs when debugging
Retrieval	Corpus bigger than the window, fast-changing sources, citations needed	Cross-document synthesis, chunking that splits a table or code block, recall misses
Compaction	Long-running sessions	Summaries drop identifiers, numbers, one-time decisions; mostly irreversible; fidelity is hard to evaluate

Practice: progressive discovery vs. monolithic — a logged weak spot

- Ask first: what share of the loaded context is relevant to a typical request? If most of it isn't (60 tools loaded, 3–5 used), **reduce what's loaded**: keep a small core set, discover the rest on demand. This lowers cost, latency, and mis-selection together.
- Don't answer by "stacking another instruction on top of what's loaded." That adds tokens while the confusing entries stay in front of the model.
- Caching lowers the cost of a repeated prefix — it doesn't fix mis-selection.
- Monolithic wins only when the set is small and almost every call uses all of it.
- Four design questions: what's needed at the start (monolithic baseline)? what's needed in the next few steps (progressive window)? what's needed on demand (retrieval)? what can be safely compacted (compaction policy)?

Extended thinking verify current docs

Reasoning in separate thinking tokens. Thinking tokens bill as output tokens and add latency. Current models use **adaptive thinking**: the model decides per-request whether and how much to think, steered by an effort level (low / medium / high / xhigh / max); `max_tokens` caps thinking plus answer together. A fixed `budget_tokens` is the older control. Raise effort only when eval shows a gap the prompt can't close; never turn it on for a routing classifier. Effort is the knob, eval is the gate.

Swapping models is a release

Gate every swap with: (1) a curated, tiered test set, (2) a scoring function, (3) a pass/fail threshold set before the run. Segment-level thresholds enable partial migration — easy segments move to the cheaper model, hard segments stay on the strong one.

1.11 Prompting as architecture D2

A reusable system prompt needs: role and scope, must-do/must-not constraints, and an output contract. Vagueness at scale is a defect repeated on every request.

Templates: parameterized slots inside a fixed frame. Guardrails live in the frame, so filling slots can't remove the constraint.

An underspecified guardrail is worse than none — it looks like a control. Prefer structural enforcement (an output contract/schema, a deterministic check) over prose.

What actually ships is the prompt-plus-model pair. Any model swap requires re-evaluating the prompt.

Technique	Use when	Stem signal
Zero-shot	Clear, scoped task; try this first	Categories already listed; standard summarization
Few-shot	Format or judgment criteria are easier shown than described	Receipts in varying layouts
Chain-of-thought	The answer depends on a reasoning path with several interacting conditions	A liability call with three interacting conditions

Every tier up adds tokens and latency. Few-shot examples need maintenance and can introduce bias if unbalanced. Distractor: adding examples or CoT to a simple task.

Prompt caching Sample Q2 pattern

The cache saves a **prefix** — it hits only when everything before the breakpoint is identical. **Order rule: static first, dynamic last.** Putting a per-request document before fixed content breaks the prefix.

Prefix byte order is `tools → system → messages`. A change at one layer invalidates that layer and everything after it. CLAUDE.md text lives inside messages.

Up to 4 breakpoints you place; each one caches everything before it. Below the model's minimum cacheable length, nothing caches — and no error is raised.

Only worthwhile when the same prefix repeats within the TTL (default 5 min, refreshed on hit; a 1-hour option also exists). Writes cost more than normal input; reads cost a small fraction. Verify before use: cache write ≈1.25× base input price (5-min TTL) or 2× (1-hour); cache read ≈0.1×; minimum cacheable length 512–4,096 tokens depending on model. Changing thinking/effort settings invalidates the message-layer cache. Confirm with `cache_creation_input_tokens` / `cache_read_input_tokens` in `usage`.

Two enablement paths: a top-level `cache_control` (automatic), or `cache_control` on the last stable content block (you choose the breakpoint).

Distractors: truncating the document, downgrading blindly, moving content into few-shot, raising `max_tokens`.

Practice: reusable prompt assets

- **Cost follows prefix order:** role, tone, output contract, guardrails = stable prefix; ticket + manual section go after the breakpoint.
- **Safety follows structure:** guardrails written into the output contract (fields the reply must satisfy), ideally plus a deterministic output check.
- **Governance follows packaging:** a stable, high-frequency, org-wide process becomes a versioned Skill. Copy-pasted prompts drift.
- **Prompt library vs. Skill:** a library suits fragments a team assembles itself; a Skill suits a stable process needing approval and rollback, distributed across teams.

1.12 Entry points, interfaces, routes, governance D3 D5 D7

Selection order: entry point (fits user and work) → build-time interface (fits team) → delivery route (fits cloud and compliance). Governance constraints exclude options before cost or convenience.

Entry point	Fits	Notes
Claude.ai (Team/Enterprise)	Knowledge workers, no dev needed	Team: admin console, SSO, no training on customer content. Enterprise: SCIM, retention controls, audit logs, Compliance API, HIPAA-ready with BAA. Cannot embed into other products
Claude Code	Engineers	Not for non-engineering workflows; never a product backend
Cowork / Chrome / Excel	Non-devs working files, web apps, spreadsheets	Permission scope and review matter more here
Your own API/SDK app	Your product, or governed internal tooling	Retries, streaming, observability, UI, audit are all on you

Build-time interface	Pick when
Direct API	Maximum control, maximum responsibility
SDK	Same capability, more ergonomic per language; default for embedding
MCP	A tool server must serve **multiple** Claude clients. One client only → an extra protocol hop and a server to build/secure for no payoff. MCP tools are still invoked via tool use — MCP and tool use are different layers, not alternatives
Agent SDK	Running a managed multi-turn agent loop inside your product (Claude Code's engine). Not for single request/response; you lose per-turn control
Agent-to-agent	Delegating to a peer agent with its own goals and reasoning. Reading data needs one tool, not another agent

Shortcut: single request/response → API or SDK. Tools shared across clients → MCP. Multi-turn agent inside a product → Agent SDK.

Claude Code's layers

Shaping — what it knows, what it does
CLAUDE.md (always loaded conventions) · Skills (loaded on demand, keeps context lean)
subagents (tool-limited isolated helpers) · MCP servers

Governing — what it can touch
hooks (deterministic gates) · permission modes (default / acceptEdits / plan / auto / dontAsk / bypassPermissions — verify latest list)
sandboxing (filesystem & network isolation)

Permission decides what an agent can touch; a hook decides what must happen before/after an action; sandboxing enforces the boundary at the environment level.

Skill as an integration mechanism added**:** mounted via a container's skills parameter, managed through the Skills endpoint, versioned like any other asset. A Skill needs the code-execution tool, which adds a sandbox dependency — weigh it against MCP and direct tool calls.

CLAUDE.md is not a system prompt. Claude Code puts Anthropic's system prompt in the `system` field and inserts CLAUDE.md into `messages`. Analogy: system prompt = the published regulation, same for everyone; CLAUDE.md = your office's desk guide. Neither is enforced. "Write it into CLAUDE.md" is a distractor for a "never allowed" rule.

Delivery routes and regulated constraints

The model behaves the same on every route. What differs: billing, identity (API key / IAM / Google credential / Entra ID), region, contract, model ID strings, and feature lag (cloud-provider routes often trail first-party).

Default to first-party with no cloud commitment or when you need the newest features; otherwise follow the partner's existing cloud agreement.

Constraint	What it decides
Privilege (law firm)	The firm must **own** the audit trail: API/SDK behind the firm's own app, SSO, and approved gateway. Enterprise logs living on Anthropic's infrastructure = an audit trail exists, not that the firm owns it
HIPAA	BAA must cover the exact config/features (beta features often excluded); minimize PHI; use reference IDs
GDPR / residency	Each request is pinned to a region. Direct API region is US or global only, so EU residency points to a cloud-provider route. GDPR itself doesn't mandate EU residency
FedRAMP / DoD	Claude for Government (FedRAMP High), Bedrock GovCloud (FedRAMP High, IL4/5), Vertex Assured Workloads. Claude Enterprise / Direct API have no FedRAMP authorization. Government environments trail on model availability
Internal policy	CIO-approved route beats engineering convenience; reuse existing SSO, data classification, logging

Stack and request lifecycle

Bottom-up: **model** (reads the request, writes text or tool **requests**) → **Claude API** (contract + server features:

caching, tool protocol, thinking, batching, billing) → **client SDK** (HTTP, auth, retries; adds no model capability) → **Agent SDK runtime** (run loop, executes tools, checks permissions, fires hooks, loads CLAUDE.md) → **application**.

The model only **requests** a tool call; the runtime **executes** it. That's why hooks and permissions live in the runtime.

The API is stateless; every turn resends the whole conversation. That's why caching matters. One user message can cost several API calls.

Loop control: `stop_reason` of `tool_use` means the runtime executes the tool and continues; `end_turn` means the loop ends.

Exam traps — Module 1

- Picking Claude Code before knowing who the user is (a bank teller won't use a terminal).
- Adding MCP for a single client "because it's on the menu."
- Putting a high-consequence check (compliance) on the least deterministic component (subagent, prompt) instead of server-side code.
- Right entry point, wrong reason (Bedrock "for residency" when the actual reason is BAA coverage) — graded as entry-point-plus-reason pairs.
- Decision order: (1) who is the user? (2) any hard governance/contract constraint? (3) real-time system integration needed? (4) latency and infrastructure. The first constraint that excludes options wins.

1.13 Assembly: five decisions every architecture answer needs

Decision	Law-firm contract review: correct answer
1. Entry point	Direct API/SDK in a lean internal web app, behind SSO and an approved gateway (privilege excludes Claude.ai)
2. Pattern	Workflow, parallel by section + a strict-schema eval step (work splits cleanly — no open-ended agent)
3. Decomposition	Claude extracts, classifies against the playbook, drafts edits. Playbook is versioned in the firm's system, retrieved by clause (not pasted into the system prompt). Pulling docs from iManage = a tool
4. Model + context	Default Sonnet, progressive context, extended thinking only on clauses eval proves need it (not Opus everywhere)
5. HITL	A senior lawyer signs off every output; low-confidence clauses are flagged

Each wrong option misses exactly one decision. Always check all five.

Module 1 self-test

1. A partner wants Claude to enforce a fixed dollar threshold. Who should own the threshold, and why does failure surface late if it doesn't?

2. Name the five pattern-selection factors, in order.
3. A parallel review run reports "all sections reviewed" but two subagents timed out. Which control is missing?
4. A user asks both "find clause 14.2" and "how do we usually handle Y?" Which index type, and how are results merged?
5. Caching isn't saving money because a per-request document sits above the fixed instructions. How do you fix it?
6. Only one internal app will ever call these tools. MCP or direct tool call? State the tradeoff.

中文要点

中文 模块 1 的核心是四个按顺序做的决策：工作分解、模式选择、参考架构、接入点与模型/上下文策略。

中文 确定性规则、实时数据、计算交给现有系统；语言工作交给 Claude；高风险决定交给人。

中文 能列举路径就用 workflow；只有路径确实无法预知时才用 agent。多 agent 汇总要做覆盖率检查。

中文 RAG 只用于稳定知识，实时状态用 tool 查询系统记录。分块看文档结构，索引看查询组合，并说明取舍。

中文 默认 Sonnet，换模型要用事先定好阈值的 eval 把关。渐进式发现减少加载量，不要叠加指令。

中文 只有一个客户端不需要 MCP。CLAUDE.md 和 Skill 都是指导，不是强制执行。合规先于成本筛选路线。

← Overview

Module 2: Enterprise Integration & Production →

MODULE 2 · 21 PAGES IN COURSE

Enterprise Integration & Production

Exam domains: D4 (largest share: eval, A/B, observability, diagnosis), D3 (auth, least privilege, observability, accuracy/latency), D1 (value pillars, feasibility), D5 (data handling, routes)

This module answers: how do you move from a working prototype to a system that clears a security review and proves every change works? Five tools: an eval gate before you build, a cost/reliability model before you ship, a feasibility conclusion before you commit, an integration design that survives security review, and an experimentation method.

2.1 Eval as acceptance criteria D4

Write the eval suite before production code. It forces measurable success criteria, exposes assumptions while changes are cheap, and gates every later change.

- Define the task: a measurable behavior, test prompts, a pass bar.
- Build a golden dataset: representative inputs plus edge cases and counterexamples, with expected outputs.
- Run automated code checks: pass/fail each.
- Score behavior needing explanation with a judge model: score + rationale each.
- Read by category, not just the average. A higher average that hides an edge-case regression is not an improvement.

Scoring method	Use for	Cost	Weakness
Code-based	Schema, JSON parsing, regex, length, exact match, numeric comparison against source, whether an action fired, identifier leakage, required sections present	Near zero; never drifts	Can't judge tone, reasoning, helpfulness
LLM-as-judge	Quality, instruction-following, reasoning, faithfulness, completeness, tone, safety handling	About one call per item	Inconsistent at edge cases
Human review	High-risk or novel behavior; calibrating the judge	Highest; sampling only	Low throughput; reviewer-to-reviewer variance

Test: does this check need explanation? Wherever a deterministic check exists, use code — **even for high-risk behavior** ("no claim auto-denied" is a code check, not a judge call).

A rigorous judge: detailed rubric, a small fixed label set, a required rationale, calibrated against human labels, run on a different model than the one it grades. An uncalibrated judge is worse than none.

Volume beats perfection: many low-cost automated cases find more regressions and can run every change.

From requirement to criteria: write field names; set thresholds from business requirements, not whatever the prototype happened to score; turn failure modes into dataset categories; add noisy and adversarial input.

Multi-turn eval is its own category: the scoring unit is the whole conversation, judged against transcript-based golden sets.

Highest-risk moment: a stale eval. The prompt changes but the golden set doesn't — tests keep passing while measuring behavior that no longer exists.

Eval runs fail loudly; production degrades gracefully. A silent fallback-model switch during an eval run mixes two models' scores. For rate limits during batch eval: respect `retry-after`, limit concurrency, or use the Message Batches API.

2.2 POC to production D4 D3

A demo hides four things: cost at real scale, latency under concurrency, reliability when calls fail, and failure modes on unexpected input. Also get an early read on business metrics as soon as possible.

- Build cost and latency models before finalizing the architecture: call volume × tokens per request (input and output) × model tier.
- Model the **token distribution**, not the average. A long tail can skew average-based estimates 2–3×.
- Design to **p95**, not the median. Example: 90 requests at 2s + 10 at 30s → average 4.8s (meets a 5s SLA) but p95 is 30s (fails it).
- Prompt caching is the strongest lever for long, stable prefixes. Risk: any part of a cached prefix that should reflect live state goes stale.

Reliability control	Handles	Sits at	Why there
Bounded retry with exponential backoff + jitter	Transient 429, timeout, 5xx/529	Right against the API call (client wrapper)	Higher up reruns the whole loop, multiplying calls
Circuit breaker	A dependency with rising error rate; fail fast	Around the service boundary for that dependency	It must see the failure rate across all requests
Fallback chain (downgrade tier, cached answer with a data-as-of date, "can't answer")	Primary model or endpoint unavailable	Orchestration layer	It's a business decision; burying it in a wrapper silently swaps models

Architecture	Fails first at	Mitigation
Agent	Unbounded tool calls, growing context; one request blows the budget	Per-turn token budget, tool-call cap, stop conditions, minimal tools; evaluate stopping behavior
RAG	Retrieval-quality drift: stale index, query/doc mismatch, outdated content	Track retrieval precision/recall as a system metric; route live-state queries elsewhere
Document pipeline	No exception path; edge-case failure rate equals clean-document success rate	Confidence scores; low-confidence → human review; edge cases in the eval set
Orchestrator-workers	Fuzzy failure boundaries, fragmented traces, silent loss	Recoverable vs. unrecoverable boundaries, shared trace ID, coverage check

Version pinning applies to every architecture: pin versions, watch deprecation pages, keep an update runbook ready.

2.3 Sizing, feasibility, ROI D1

Three sequential questions, each feeding the next, all landing in the SOW: **can we afford to run it? can we build it? is it worth it?**

Sizing

- Get call volume from the business owner, not sampled from logs. Count follow-up calls in multi-turn conversations.
- Tokens per request computed as a distribution.
- Monthly cost = input tokens × input price + output tokens × output price, cached tokens at cache-read price, plus cache-write cost. Compare against the budget cap. If the SLA allows async, price the Batch API discount (confirm BAA coverage for regulated data).
- Sensitivity analysis: double the call volume; shift tokens toward the long tail.
- Over budget → change the architecture before writing code (narrow scope, cheaper tier, caching, batching).

Feasibility

- Scoping: business goal → capability list → owner per capability → boundary conditions → SOW scope clauses.
- Check every capability against the four model properties. **Working memory is the one most often overlooked:** a single long document may fit; the whole corpus never will — exactly why RAG exists.

Conclusion	You must deliver
Feasible as scoped	Written assumptions (the conclusion changes if an assumption changes)
Feasible with constraints	Each constraint and its failure mode if violated — the constraint is itself architecture
Not feasible	The binding constraint, and what scope reduction would flip the conclusion

Practice: name the one load-bearing constraint

- Research assistant, 50 reports/week, 24h delivery, $500/mo → feasible as scoped; no binding constraint. Adding full human review everywhere is not the binding constraint.
- Carrier-email extractor, 5,000/day, writes to the order system → feasible with constraints: extraction accuracy before a transactional write → code-based eval + a human gate on low-confidence writes.
- Trading advice within 2 seconds → conclusion pending: verify the live-quote tool + model round trip can fit in 2s first. A human gate doesn't fit inside 2 seconds.

Rule of thumb: write-side errors with consequences → accuracy eval + HITL. Real-time data → a tool + a latency budget. An unknown that decides the conclusion → a technical spike first.

ROI by five value pillars

Pillars: **efficiency, transformation, productivity, solution cost, performance SLAs.**

- Baseline in business units, measured from the owner's real data.
- Expected state in the same unit, keeping residual HITL labor.
- Subtract run cost from the sizing model (distribution, not average).
- Payback period, including build cost, with a sensitivity check.

Three classic mistakes: an estimated baseline instead of a measured one; HITL designed in but modeled as fully automated; run cost computed off the average.

Scope vs. sizing vs. feasibility vs. reliability: scope = what (qualitative); sizing = run cost; feasibility = can the design meet the bar; reliability = can it survive infrastructure failure in production. Compensating controls for model limitations (schema, retrieval, HITL) are not reliability controls (retry, breaker, fallback).

2.4 Enterprise integration patterns D3 D5

Order: compliance filters routes and entry points first, then identity, authorization, data handling, observability.

Layer	Decision	Failure if wrong	Rule
1. Compliance	Which routes/entry points survive	Fails legal/security review; rebuild	Authorization covers a configuration, not a vendor; approved beats convenient
2. Identity/SSO	Where identity is verified; how it enters the prompt	A role self-declared in a user message can be spoofed	Verify server-side before the call; the auth layer injects role and permissions
3. Authorization	Claude follows the existing access model	A user reaches data they're not entitled to	Enforce at the tool/data layer, scope from the server session, filter before retrieval
4. Data/PII	Which fields enter context	PII appears in plaintext in your own request logs	Context is not a governance boundary. Minimize fields, use reference IDs, redact before the call
5. Observability	What must be reconstructable after an incident	An unlogged path is invisible; security won't allow an agent to act	Log request, response, context, and outcome on every call

Log four things per request: request (model version, input tokens, prompt ID); response (output tokens, latency, stop reason); context (user role, session ID, cache hit); outcome (did downstream accept it? rejection signal?).

Tool least privilege: review each tool as required vs. merely convenient; remove out-of-scope tools and record why; scope each subagent's tools. Removal beats logging or confirmation (Sample Q1).

Capability-bloat test: does each tool serve a task users actually do? Removal closes read risk, write risk, and future misuse at once. Check whether your reasoning matches the specific risk the stem names (read of unauthenticated data vs. write).

Multi-tenant: one API key per tenant for rate-limit attribution and isolation.

Data retention added**:** Anthropic doesn't retain API content by default, but your own application logs record every field you sent. Redact before the call; redacting logs after the fact is the most expensive fix.

Entry points from an integration lens: Claude Code can never be a multi-tenant product's backend; Agent SDK doesn't fit single request/response; MCP adds a protocol hop that makes tool debugging harder.

Constraint	Route / integration
Attorney-client privilege	Firm's own app + gateway holds the keys, enforces access control, owns the audit log
HIPAA	BAA covers the exact configuration; minimum necessary; log model version, user, data scope
GDPR / EU residency	Pin region per request; EU residency → cloud-provider route (Direct API is US or global only)
FedRAMP	Only Claude for Government, Bedrock GovCloud, or Vertex Assured Workloads. Holding a Claude Enterprise license ≠ compliant
Internal policy	CIO-approved cloud; reuse SSO, data classification, logging

2.5 A/B testing, shadow testing, observability D4

Experiment requirements — all set before the run

- A falsifiable hypothesis: treatment, **one primary metric**, minimum effect, guardrail metrics.
- Randomized, consistent per user or session.
- Sample size depends on baseline, minimum detectable effect, power, significance. LLM variance means larger samples (course example: detecting 70%→75% at 80% power / 5% significance needs ≈1,500 per arm).
- Choosing a metric after seeing the data is outcome-shopping.

Practice: the experiment design plane — a logged weak spot

Keep two axes separate. **Effect size** = how large the measured-metric difference is. **Confidence** = the cost of being wrong (consequence, reversibility) — driven by risk, independent of effect size.

- Sample size rises with confidence, falls with effect size; halving the effect roughly quadruples the sample.
- Medical intake summarizer, minor change → small effect / **high** confidence (largest, slowest sample).
- Switching a formatting task from Sonnet to Haiku, cost change big and reversible → large effect / **moderate** confidence; watch quality guardrails.
- Adding a routing category that takes 30% of traffic → large / high (blast radius). FAQ wording tweak → small / low.

Reading results: statistically significant ≠ practically significant. Confirm the gain is worth the maintenance and that no guardrail metric (cost, p95) got worse. **Even an overall improvement with a high-risk segment regression is a guardrail failure** (overall 91%→93% while a key segment drops 85%→70% = don't ship). Register segment-level guardrails before the run.

Trap: a few dozen sessions per arm, a few points' improvement, uncontrolled input mix, metric chosen after the fact. That's noise.

Shadow testing: mirror live traffic to the new version, serve users only the current version, score the shadow output offline. Fits when a single bad output is unacceptable (regulated) or traffic is too low. You lose real user signal and depend on a rubric or gold answers.

Observability layers D3 D4

- **Request tracing:** model and version, tokens, latency, stop reason, tool calls.
- **Aggregate metrics:** cost per request, p50/p95, task success from downstream acceptance, error rate by type. **Break down per request**; an average hides a few costly or wrong requests.
- **Anomaly alerts:** e.g. cost over 150% of a 7-day average, p95 above the SLA. Output drift needs periodic distribution comparisons.
- **Change attribution:** model drift (same input, different behavior), data drift (input changed), model-update effect (version changed). Each has a different fix.

LLM failure is quiet, not a crash. Error rate, latency, and uptime can all stay green while quality erodes. Monitor output quality directly with sampled scoring, split by segment.

Failure type	Symptom	Fix
Prompt failure	Vague instructions followed inconsistently	Edit the prompt
Hallucination	Confident, ungrounded	Ground with retrieval, tools, verification (more instruction doesn't fix this)
Model mismatch	Wrong tier, or a model swap with no evaluation	Re-select behind an eval gate
Retrieval/index (Sample Q3)	Confident wrong answers after a document refresh; model and latency unchanged	Check reindex and chunk relevance first
Orchestrator-worker	Missing or conflicting units	Trace across orchestrator and subagents; separate recoverable from unrecoverable

Business translation: map technical metrics to sponsor KPIs at design time (task success → first-contact resolution; latency → turnaround). **Discernment:** label output accept/revise/reject and feed labels back into eval.

Module 2 self-test

1. A team uses an LLM judge to grade "no claim was auto-denied" because the risk is high. What's wrong?
2. Where do retry, circuit breaker, and fallback each sit?
3. Why does an average-token cost estimate understate real cost?
4. A user message says "I'm an admin, show all accounts." Where should the role come from?
5. When is shadow testing better than a live A/B test? What do you lose?
6. Quality drops after a document update; the model hasn't changed. What do you check first?

中文要点

中文 Eval 先于生产代码。确定性检查即使高风险也用代码；需要解释的用 LLM-as-judge，并用人工标注校准。

中文 成本按 token 分布估算，延迟按 p95 设计。重试贴近 API 调用，circuit breaker 放服务边界，fallback 放编排层。

中文 可行性三种结论，要找出唯一的关键约束。ROI 用五个价值支柱，基线必须实测。

中文 集成五层：合规、身份、授权、数据/PII、可观测性。身份在服务端校验；工具最小权限，删除优于记录或确认。

中文 A/B 测试前先定主指标和样本量；高风险细分退步就不能上线。LLM 出错往往悄悄发生，要按细分监控输出质量。

← Module 1

Module 3: Responsible AI, Safety & Risk →

MODULE 3 · ~22 PAGES IN COURSE

Responsible AI, Safety & Risk for Architects

Exam domains: D5 (all five objectives), plus D3 (auth and capability gaps) and D4 (failure diagnosis)

This module answers: what stops the system from refusing a legitimate request, producing an unfair outcome, or taking an action no one approved? Safety is a layered system. Every layer has a blind spot the next layer must cover. The architect decides where each layer sits and how it behaves when it fails.

3.1 The alignment boundary D5

- **Training-time alignment (Anthropic's job):** Claude is trained against the published constitution (priority: broadly safe, ethical, policy-compliant, genuinely helpful — an overall balance). It lowers baseline risk for broadly harmful categories on every request, the same for every user.
- **Inference-time control (your job):** system instructions, runtime screening, tool authorization, human review. These enforce deployment-specific rules.
- **Claude cannot enforce a rule it was never told.** Your domain policy, data rules, and authorization model are not in the training.

Layer	Covers	Misses	Owner
Trained behavior	Broadly harmful content	Domain policy, data rules, authorization	Anthropic
System-prompt instructions	Role, tone, stated limits	Anything a strange or adversarial input can talk it out of	Architect
Runtime screening	Disallowed content in input/output	Actions with side effects; novel attacks	Architect
Authorization	Can this caller do this now	Content quality and fairness	Architect

Exam trap

"Claude refused every harmful test prompt, so it will also block cross-tenant access." Wrong. A normal-looking in-domain request that violates a policy exists in none of those layers. Broad harm → trained behavior. Tenant isolation, approved phrasing, refund sign-off, program eligibility → the application layer.

3.2 Risk categories and assessment D5

Risk	What happens	Control sits at
Direct prompt injection	User input overrides instructions	Input screening; structure over instruction
Indirect prompt injection	Instructions hidden in retrieved documents or tool output treated as trusted content. **The primary attack path for RAG and agents**; invisible to user-input screening entirely	Screen retrieved content and tool output before adding it to context
Token-budget exhaustion	Padded input truncates work or spikes cost	Input limits, budgets
Tool and action abuse	Model is induced to call a side-effecting tool outside policy	Deterministic tool authorization; least privilege
Data exposure	Sensitive fields enter context or logs regardless of model behavior	Minimize and redact before the call

Practice: a written risk assessment

Walk the request path and the data path together. At every entry point (user input, retrieved content, tool output, model output, logs), ask: what could an attacker do? which control stops it? A plausible attack with no matching control is a finding. Log per risk: category, affected component, likelihood and impact, mitigating control, owner, evidence. A security reviewer signs this document.

3.3 Placing guardrails on the request path D5 D3

Three decision points, each answering a different question. A control at one point does nothing for the others — an output filter at the end can't stop a refund tool that already fired.

Control point	Question	Use a model when	Use a deterministic rule when
1. Input screening	Should this request reach the model?	Ambiguous intent, jailbreak/injection patterns	Blocklist, regex, length, format
2. Output screening	Can this output be returned safely?	Toxicity, policy compliance (judge)	Known strings, disallowed fields, schema violations
3. Tool-call authorization	Can this caller do this action in this context?	Rarely	**Almost always:** allowlist + identity + scope, provable and replayable

A classifier can be talked around; a rule is brittle. Chain them and write down what each one misses.

Fail closed everywhere a wrong pass-through causes harm. If you don't choose a direction, surrounding code usually lets traffic through — a guardrail that silently fails open is worse than none. Log every block or failure at a gate. This applies to components you build yourself; Anthropic's built-in safety controls can't be configured by the operator added.

Refusals verify docs**:** the API returns `stop_reason: "refusal"`, possibly with a nullable details object. Handle it per category and reset or rewrite the refused turn; resending on the refused context keeps getting refused.

Reference path: request → input screening (fail closed) → model → output screening (fail closed) → deterministic authorization before any side effect → response, logged at every gate.

Skill supply-chain security

- A Skill is distributed code plus instructions; malicious logic in the package is invisible to conversation-level screening.
- Audit before trusting: network, shell, filesystem, credential reads, behavior beyond the stated purpose.
- Runtime isolation as a safety net: least privilege, sandboxing, limited file/network access, no standing credentials.
- Trusted-source policy: internal repos, verified publishers, signed releases. Don't assume the platform vets a Skill for you.
- Record every audit outcome: approved, rejected, or re-audited after remediation.

3.4 Fairness and transparency D5

Fairness is an architectural property with four injection points you control:

- **Retrieval corpus:** over- or under-representation, outdated or single-region guidance.
- **Prompt framing:** embedded assumptions ("flag suspicious applications" nudges toward denial).
- **Few-shot examples:** same skew risk as the corpus.
- **Downstream routing:** who gets human review, who gets auto-decided.

Audience	Needs from the same decision record
Affected user	Why the decision happened, in actionable terms (a concrete denial reason)
Regulator	Consistent handling of comparable cases; ability to reconstruct any decision
Dev team	Full trace: prompt version, retrieved content, output, every routing step

Decision logging: input, retrieved document IDs, prompt version, example set, output, routing path — indexed so a single decision can be replayed. Build it once; every audience draws from it.

Gaps to find: an unlogged routing step, retrieved content that was never captured, outputs stored with no inputs, overall accuracy with no sub-group breakdown. The decision log is itself sensitive data — minimize, limit retention, control access.

Exam trap

"The vendor ran a bias evaluation" or "the model was trained to be fair, so fairness is handled." Your corpus, framing, examples, and routing create skew the vendor never saw. Instrument and monitor it; don't assume it's a built-in model property.

3.5 Human-review routing by stakes D5

Treat reviewer attention as a fixed budget. **Reversibility and error cost set the risk tier. Confidence** estimates how likely an output is wrong; it sets volume, not risk — and it's only useful when calibrated.

Rule: low confidence **and** irreversible or high-cost → route to a human. When signals conflict, weight cost and reversibility more heavily. Route by risk, not by volume.

Placement	Gives you	Costs	Fits
Pre-action approval	Nothing irreversible happens unreviewed	Latency; depends on reviewer availability; doesn't scale	High-risk, irreversible (a low-confidence denial)
Post-action audit	High throughput	The error already happened	Reversible, lower-cost
Sampled review	Monitors quality without slowing the flow	Individual bad cases can slip through	Monitoring the whole system

The reviewer's interface determines whether review is real: show the input, the model output, and the specific reason it was flagged.

Two independent failures, either one disables review: too much volume (consent fatigue, review becomes rubber-stamping) and missing context. Anthropic's own agentic research leans toward fewer step-by-step approvals and review concentrated at higher-value checkpoints (plan review, exceptions) — which is why Claude Code has plan-level approval.

3.6 Compliance: from obligation to evidenced controls D5

The route is a prerequisite, not proof. Every obligation becomes three things: a specific technical control, a named owner, and continuously updated evidence (the most commonly missing piece).

Obligation	Control	Evidence	Owner
HIPAA PHI	HIPAA-ready plan or a BAA'd API; HIPAA settings on; only in-scope features	Signed BAA, admin-console config screenshot, compliant-feature list	Security lead
FedRAMP impact level	An authorized route meeting or exceeding the required level	Authorization record; proof the workload only runs in that environment	Platform lead
Data residency	In-region processing and storage, logs/cache/monitoring included	Residency config + a data-flow record of every copy	Data owner
Decisions reconstructable	Retained, queryable decision log	A reconstructed example from live logs	Architect

Not used for training ≠ not retained. Data can be excluded from training and still be retained for logging, abuse prevention, or legal reasons.

Reviewers accept signed agreements, config screenshots, authorization records, or a log query that returns a result. They don't accept a design doc that names a control with no owner and no evidence (same logic as FIAR KSDs and A-123 testing).

Configurations drift; revalidate the register periodically. "A prompt instruction to handle health data carefully" is not HIPAA evidence. A BAA plus an enabled configuration is.

3.7 Assembling the safety stack (order matters)

1. Draw the boundary: domain rules in the application layer; explicit rules into a deterministic rules engine; Claude drafts the explanation.
2. Place input, output, and authorization controls; pick a check type for each; fail closed.
3. Instrument the four fairness injection points, serving all audiences from one decision log.
4. Route review by risk; pre-action approval for low-confidence, hard-to-reverse harm.
5. Build the control register: obligation → control → owner → evidence.

Steps 3 and 4 depend on logging; step 5 depends on evidence from steps 2–4.

Your logged weak spots — Module 3

- **Implement guardrails and safety controls:** a Skill or prompt can't enforce; a hook or authorization can. Scope comes from the server-side session, never from a model-supplied tool parameter. Watch your pull toward detective controls (trace IDs, logging) when a preventive option is available.
- **Ensure compliance with regulations:** filter by authorization first. For CUI/FedRAMP, Claude Enterprise is excluded regardless of license count. Reason from the exam's rule, not "DoD already uses Bedrock."
- **Identify risks, limitations, failure modes:** drill the four model properties until they're reflexive.

Module 3 self-test

1. Which layer enforces "never show tenant A's data to tenant B"? Why can't trained behavior do it?
2. A RAG agent executes instructions hidden in a retrieved page. What risk is this, and where does the control sit?
3. Name the three components of deterministic tool authorization.
4. Your screening service times out at peak load and requests pass through anyway. What's wrong?
5. A decision is high-confidence but irreversible and high-cost — route to a human? What variables decide it?
6. What three things does every obligation in the register need?

中文要点

中文 安全是分层体系。训练对齐只覆盖广泛有害内容；领域规则、租户隔离、授权必须在应用层实现。

中文 间接 prompt injection 来自检索内容和工具输出，是 RAG/agent 的主要威胁，要在加入上下文前筛查。

中文 三个控制点：输入筛查、输出筛查、工具调用授权。授权必须确定性（allowlist + 身份 + 范围）。自建护栏要 fail closed。

中文 公平的四个注入点：语料、prompt 措辞、示例、下游路由。一份决策日志服务用户、监管、开发三类受众。

中文 人工审核按风险（可逆性 + 错误成本）路由，置信度只决定数量。每项合规义务要有控制、负责人、证据。

← Module 2

Module 4: Stakeholder Communication & Lifecycle →

MODULE 4 · ~20 PAGES IN COURSE

Stakeholder Engagement, Lifecycle & GTM

Exam domains: D6 (all five objectives), plus D1 (value pillars) and D3/D5 (route selection, residency)

Lifecycle framework running through this module: discovery → design → handoff → monitoring → iteration. Knowing which phase a decision belongs to tells you when that phase is allowed to close.

> **Scope note:** demo design, joint scoping with Anthropic's Applied AI team, GTM partnership maps, and the "reuse potential" outcome-doc field are all **Partner Track, out of scope for CCAR-P**. Selecting the reuse-potential field is always wrong on this exam.

4.1 Discovery is structured elicitation D6

Three steps: listen to the business goal; translate it into requirements, assumptions, and open constraints; write each one down before the conversation moves on.

Category	Ask	Note
Must do	Capabilities stated as business outcomes	This is where you split Claude / system / human
Must not do	Prohibited actions; cases that must go to a human	**Stakeholders rarely volunteer this — ask for it**
Must cost	Latency targets, per-interaction cost ceiling, call-volume forecast	Feeds sizing and SLAs
Must prove	Evidence obligations, especially regulated settings	Cheaper to find now than at legal review

Translation is the core skill. Experience words ("seamless," "fast," "simple," "just a quick check") are a signal to dig deeper. Ask: what would break that experience? what must still hold when something goes wrong? Answers become testable constraints (a p95 target, a handoff rule, a safe failure state).

Output: a translation table, one row per item: their words → implied constraint → architecture decision → assumption to confirm.

- "It just reads the form and routes it" → routing is very likely a deterministic rule.
- "The reviewer will check it anyway" → very likely a mandatory authorization gate.
- "We're healthcare, handle data carefully" → a formal evidence obligation; audit trail and data handling are requirements from day one.

Requirement vs. assumption: a requirement traces to something a stakeholder said. An assumption traces to nothing — an untraceable assumption is the most dangerous kind.

Exam trap

The architect proposes a design mid-call, the stakeholder agrees, questions stop, and a hidden constraint surfaces weeks later. Don't propose a solution before all four question categories are asked.

4.2 Framing tradeoffs D6

The goal is an informed decision by the other side, not a conclusion reached for them. Every tradeoff states:

- What we get.
- What we give up.
- **The cost of reversing the choice once the system is built around it** (most readouts skip this).
- In regulated settings, the effect on compliance posture.

Deliver a **decision package:** options considered, evaluation criteria, recommendation, residual risk. The stakeholder has to defend this decision to their own leadership. Lead with business outcomes; state limitations honestly.

Tradeoff	Get	Give up / reversal cost
Stuffing context per call vs. retrieval	Simpler at first	Per-call cost and latency rise at scale; reversing means rearchitecting after costs spiked
Cutting logging for lower latency	Smoother UX	Visibility drops; a compliance gap in regulated settings
Single route vs. multi-platform	Simpler auth and logging	Less flexibility on region/procurement; a later residency requirement can get stuck switching

Exam trap

A correct answer to the question asked (per-call cost) is not the same as informed approval — not if no one converted it to monthly cost at production scale, and no one explained the cost of reversing the choice.

4.3 Feedback loops and SLAs D6

Deployments drift quietly. **A feedback loop is a decision layer built on top of observability:** Signals → Triage → Decide → Act → Review.

An SLA states: what's measured, what counts as breach, what breach triggers. **Every threshold must trace to a source:** latency → the experience expectation from discovery; availability → business criticality; quality → eval acceptance criteria.

Cost is the most commonly broken expectation. Production scale is often one to two orders of magnitude above

the pilot. Give the sponsor a real-scale usage forecast and a cost-control plan (caching, model tiering, budget alerts) before the first bill lands.

Regulated deployments need calendar-triggered checkpoints even when every metric is green (periodic output audits, scheduled residency confirmation). **Escalation triggers are predefined** (an agreed threshold or a calendar date), not reactive.

Triage category	Criterion	Example
Noise	Nothing changed, or a one-time event that self-resolved / has a known cause	One malformed request from a known buggy client; a successful 2am retry; error rate flat after a new template shipped
Internal monitoring	Something changed, but within budget or explained	p99 up 40ms but still within budget; token use up and matching known seasonal peak
Architect review	A persistent trend that raises a design question	Eval score declining three weeks running
Stakeholder review	Crosses a line the stakeholder owns, or a planned regulated checkpoint	Per-interaction cost over the agreed threshold; a scheduled residency confirmation; a quarterly output audit

Exam trap

Mistaking strong observability for a feedback loop. A slowly declining eval score never trips an error-rate alert; with no trend trigger and no owner, the problem surfaces months later. Set triggers for slow drift. Also: error rate is the wrong metric for a prompt change; eval checks belong in the pre-release gate.

4.4 Documentation for handoff and audit D6

Reader	Needs
Incoming engineer (successor)	Decisions plus **rejected options and why**, or they'll reopen a load-bearing choice
Compliance reviewer	Obligation → control → owner → evidence. No verbal assertions accepted
A future architect returning later	Dated decisions, assumptions explicitly labeled, open items with owners

Completeness test: could a competent architect who wasn't present safely modify the system after reading this? Checklist fields: decision (dated), rejected alternatives, tradeoff (get/cost/reversal), owner, evidence, audit readiness.

Practice: placing documentation artifacts — a logged weak spot

Two tests: (1) who's the **primary** reader, successor or reviewer? (2) does it record **reasoning/intent**, or **proof** of what happened?

- **Handoff / intent:** architecture diagram, assumptions register, **decision log with rationale** (a successor

reads this first, to avoid reopening a settled decision).

- **Handoff / evidence:** deployment runbook.
- **Compliance / evidence:** control register with evidence links, test-result summary.
- **Compliance / intent:** empty. Not every quadrant needs an entry.

Government-practice trap: in FIAR, a documented management rationale counts as audit evidence, which pulls the decision log toward the compliance side. The course classifies by primary reader and function instead.

Exam trap

A detailed architecture diagram with no stated rationale. A successor changes the context strategy for performance and silently breaks a data-residency constraint. A diagram shows "what," not "why."

4.5 Entry points and outcome documentation at scale D6 D3 D5

Multi-platform traps: model ID strings differ by route; cloud-provider routes lag on features; Bedrock and Vertex require explicit region configuration (accepting a default or global endpoint is the classic residency violation).

Write an entry-point responsibility map before writing integration code: which route owns which task, and why. It stops a route chosen for one job from silently taking on another.

Route	Pick when
Direct API	Newest features arrive first, fewest hops. Default unless procurement or residency points elsewhere
Bedrock	Partner standardized on AWS, needs in-region execution; requires explicit region config
Vertex	Partner standardized on GCP with a Vertex procurement channel
Foundry	Behavior depends on hosting form; verify residency per model

Practice: primary vs. secondary route — a logged weak spot

- **Procurement and data residency decide the primary route first,** ahead of cost or latency. AWS-standardized + a Moderate-level in-region obligation → Bedrock is primary.
- **Latency decides what the secondary route is used for — never the primary.** Direct API carries latency-sensitive, unregulated work.
- "None required" for a secondary route is correct only when nothing requires lower latency than the primary provides.
- Principle: governance constraints exclude options first (pass/fail); cost and convenience only decide among what's left.

Outcome-doc field	Purpose

1. Use case & scope boundary	What changed
2. Metric before	Baseline captured at the start
3. Metric after	Same metric, post-launch, same definition
4. Auditable control	What makes the comparison credible (e.g. an authorization log)
5. Measurement owner	Who keeps measuring after the project ends
6. [Partner Track] Reuse potential	Never correct on this exam

Pick fields by the question asked: a reusable value story → before + after metrics. Compliance sign-off → control + owner.

Trap: writing the outcome doc from easily exported technical metrics (call volume, latency, error rate). That proves the system ran, not that it changed anything. A CFO needs before/after business metrics backed by an auditable control.

Phase gating: the outcome doc decides whether to expand. Early data is often insufficient — name an owner, confirm the control is logging, and schedule completion at a milestone rather than publishing a premature conclusion.

4.6 Worked example: seven decisions for a regulated, multi-platform rollout

1. **Discovery:** a "must prove" constraint (every clinician-authorized output has an audit trail) becomes a requirement.
2. **Tradeoff:** cutting logging for latency — state the gain, the loss, the reversal cost, the compliance risk.
3. **Feedback loop:** a calendar-triggered audit rule, owned by compliance, independent of metrics.
4. **Documentation:** a decision-log entry on explicit in-region routing, naming the rejected global endpoint and why.
5. **Entry point:** primary = the partner's standardized cloud route with an explicit region; secondary = Direct API for unregulated work.
6. **Outcome doc:** a baseline cycle-time metric, the same metric post-launch, an authorization log as the control.
7. **Phase gate:** is post-launch data sufficient? If not, name an owner and a completion date.

Your logged weak spots — Module 4

- **Document architectures and provide implementation guidance:** classify by primary reader and function; never select "reuse potential."
- **Manage stakeholder feedback loops:** internal monitoring requires a real change — no change = noise.
- Government-practice note: your governance background is a real strength, but the exam's framework sometimes differs (decision log = handoff, not compliance).

Module 4 self-test

1. Name discovery's four question categories. Which one do stakeholders rarely volunteer?
2. What four elements belong in a tradeoff readout? Which is usually missing?
3. A metric has declined for six weeks with no alert. What's missing?
4. Where should latency, availability, and quality thresholds each come from?
5. Which two outcome-doc fields answer "can this be reused as a value story"?
6. What configuration mistake commonly breaks data residency on a cloud-provider route?

中文要点

中文 Discovery 用四类问题：必须做、不能做、成本、必须证明。"不能做"利益相关者很少主动说，要主动问。问完之前不要给方案。

中文 Tradeoff 要讲：得到什么、放弃什么、事后反悔的代价（最常漏）、对合规的影响。

中文 反馈闭环是 observability 之上的决策层。SLA 阈值要有来源；受监管场景要有按日历触发的审计；缓慢漂移也要有触发器。

中文 文档按主要读者和功能归类：决策日志属于交接/意图，控制登记表属于合规/证据。

中文 主路线由采购和数据驻留决定，延迟只决定次路线的用途。成果文档要有前后业务指标和可审计控制；reuse potential 永远不选。

← Module 3

Module 5: Team Enablement & Ops Productivity →

MODULE 5 · ~10 PAGES IN COURSE

Team Enablement & Operational Productivity

Exam domain: D7 (all three objectives, 7%), plus part of D4 (diagnosis)

This module answers: the system is built; how does the team use it well, and keep it healthy, without the architect in every issue? Three steps in order: configure the environment, lift the daily workflow, sustain operations.

5.1 Team setup

D7 · Configure Claude tools and environments

- **Environment = shared configuration.** For Claude Code: one project-level baseline — shared CLAUDE.md, agreed tools and MCP servers, agreed permission settings. Review, version, and improve once, instead of reconciling dozens of drifting personal configs.
- **Rollout = champion first, then waves.** One champion per department migrates a real workflow first, absorbs early friction, builds local examples, tunes the shared CLAUDE.md, and becomes frontline support. Then roll out in small peer batches. Turning everyone on at once creates confusion and a retreat to old habits.
- **Set a cost policy before the first bill:** default model, model allowlist, effort guidance, and per-person cost/rate caps. Ungoverned model choice multiplies across every member and every request.

Skill distribution method	Best fits	Governance & rollback
Org-provisioned Skill	Everyone in the org should have it	Owner controls availability; can't pin a version, no native rollback (manual re-upload only). Weakest rollback
Plugin assigned to a user group or the whole org	A specific team, or anything needing controlled rollout	Group targeting, install preference (required / default-install / optional / unavailable — added), version-controlled updates from a repo. **Strongest people-facing governance**
Claude Code project Skill (.claude/skills/)	One team's coding convention across its own repos	Versioned and rolled back via git; scoped only to the repo that contains it
API Skill (Messages API container)	A partner's own product calling it programmatically	Governed in the calling system; explicitly pinned version; machine-to-machine

Practice: the Skill-distribution decision order

1. Caller is a program, not a person? → **API Skill.** Decided before governance even enters.
2. The artifact lives in a repo as a coding asset? → **Project Skill.** Rollback is inherited from git.
3. Otherwise, for people teams: does it need centralized versioning and rollback, whatever the audience size? Yes → **plugin.** No, and it genuinely applies to everyone → **org-provisioned Skill.**

Audience size alone never decides step 3 — two questions about "everyone" can have opposite correct answers. Centrally managed Claude Code settings are a separate settings channel, not a Skill-distribution path — a common distractor.

Exam trap

Pushing a shared Skill as a flat package with no version and no rollback. One bad edit propagates to every team, producing bad output continuously during a manual fix. Any shared asset needs a version, an owner, and a rollback path.

5.2 Developer workflows D7 · Improve developer workflows

Put assistance inside the existing workflow (editor, code review, test loop), not in a side chat window. Encode conventions and repeated processes as Skills and project configuration.

Stage	Claude helps with	Review discipline still required
Write code	Boilerplate, tests, a first implementation from a clear spec	Correctness and safety review; the author understands what was generated
Review code	Diff summaries, flagged possible issues, explaining unfamiliar code	The human decides; an AI flag is input, not a conclusion
Debug	Hypotheses from symptoms and traces	Verify with evidence before acting

Two adoption failures: uneven adoption (a few heavy users, many non-users — fixed with champion + wave rollout), and staying at simple chat (never progressing to tool use, codebase-aware help, Skills — having permission isn't real adoption).

Diligence = a verification checklist across four dimensions: **correctness, security, maintainability, human understanding** (the person merging can explain what the code does, including on untested inputs). Automate checklist items into tests and evals every change must pass.

Exam trap

Tests are green, review is approved, but the author can't explain how one input is handled. Pause the merge. The missing control isn't a linter or a faster model — it's the checklist's "understanding" and "security" dimensions.

5.3 Operational support D7 · Support debugging D4 · Diagnose

The architect's job is translating symptoms into architectural root causes, and teaching that path to the team so they handle it themselves next time. Fixing every incident personally is firefighting, not enablement.

Symptom	Likely cause	First action
Quality declining gradually, no code change	Model or prompt changed, or corpus growth caused retrieval drift	Rerun the eval set; check what changed in model, prompt, corpus
Latency spiking	Context grew, a tool slowed down, or caching stopped hitting	Trace the slowest link: tokens per request, slowest tool call, cache behavior
A tool fails intermittently	Authorization, rate limiting, or an unhandled error path	Check that tool's auth and rate limits; trace one failed call end to end
Cost rising with flat usage	Model tier silently escalated, or caching degraded	Check per-request tier and cache-hit rate against the budget model

Autonomy is built, not granted: a runbook (symptom → cause → action) and an escalation path (who owns what, when it leaves the team). Goal: the team only needs you for genuinely new problems.

Exam trap

Dashboards are green all quarter while answer quality declines, because corpus growth outpaced the index and the runbook has no entry for "quality declining, no code change." Raising the model tier, disabling the cache, or rolling back the last deploy are distractors. Check model, prompt, and retrieval drift first.

Your logged weak spot — Module 5

Configure Claude tools and environments: the relationship between CLAUDE.md and the system prompt. System prompt = Anthropic's, in the `system` field (the published regulation). CLAUDE.md = your repo's desk procedure, inserted into `messages`. Neither is enforced — both are guidance.

Module 5 self-test

1. Which Skill-distribution method gives group targeting, versioned updates, and rollback all at once?
2. Why doesn't an org-provisioned Skill fit content that needs fast rollback?
3. Name four cost-policy controls.
4. Name the checklist's four dimensions. Which one catches "no one can explain this code"?
5. Traffic is flat but cost doubled. What two things do you check first?

中文要点

中文 团队配置：共享基线（CLAUDE.md、工具与 MCP、权限）；先每部门一个 champion，再分批推广；上线前设好成本控制。

中文 Skill 分发顺序：程序调用 → API Skill；代码库资产 → 项目 Skill；需要集中版本控制和回滚 → plugin；全员且不需版本 → 组织级 Skill。

中文 开发流程：AI 嵌入现有流程；验证清单四维度：正确性、安全性、可维护性、提交者能解释代码。

中文 运维：症状 → 原因 → 首个动作；质量缓慢下降先查模型、prompt、检索漂移。留下 runbook 和升级路径。

← Module 4

One-page Cheat Sheet →

SECTION 7 EQUIVALENT · THE ONLY PAGE TO RE-READ 12:00–1:30 THE DAY BEFORE

One-Page Cheat Sheet: Stem Signal → Answer

If the stem says ... → lean toward ...

Agent has a tool its role never needs	Remove it (not logging, not a confirmation, not a bigger model)
A rule is already in the prompt and still fails	Deterministic control: hook, authorization, schema. Not more instruction, not a Skill
Threshold, eligibility, priority, coverage rules	Deterministic code / rules engine; Claude extracts and narrates
Order status, inventory, balance, queue + retrieval	Replace retrieval with a tool call to the system of record
Same large static prompt every call; cost and latency	Static first, dynamic last; enable prompt caching
60 tools loaded, only 3–5 used; slow, expensive, mis-picks	Progressive discovery: a small core set, the rest on demand
Only one client will use this integration	Direct tool call or API, not MCP
Multiple Claude clients need the same tool set	MCP
Path is enumerable, but the stakeholder wants "flexibility"	Workflow (routing/chaining/parallel/evaluator-optimizer), not agent
Parallel aggregation claims completeness but a unit is missing	Coverage check at aggregation; retry and flag the gap
Which model?	The smallest tier that clears the eval bar; default Sonnet; tier per step
Confident wrong answers after a document update	Check retrieval/index first (stale or mismatched chunks)
Infra dashboard all green, users say answers are worse	Sampled output-quality scoring, split by segment
High-risk deterministic check	Code-based scoring, not LLM-as-judge
Overall improved, but a high-risk segment regressed	Don't ship; segment guardrail failed
Small-sample A/B, metric chosen after the fact	Noise; register metric and sample size before the run
One wrong output is unacceptable, or	Shadow testing

traffic is very low	
429s during an eval run	Backoff honoring retry-after, limit concurrency, or Batches API — never silently downgrade
A user message self-declares a role	Server-side identity; scope from the session; enforce at the tool/data layer
PII in request logs	Minimize and redact before the call; use reference IDs
CUI / FedRAMP workload	Only Claude for Government, Bedrock GovCloud, or Vertex Assured Workloads
EU data-residency requirement	A cloud-provider route with an explicit region (Direct API is US or global only)
Attorney-client privilege, the firm must own the audit trail	API/SDK behind the firm's own app and gateway
Instructions hidden in retrieved documents or tool output	Indirect prompt injection: screen retrieved content before it enters context
A screening service times out, traffic passes through anyway	Fail closed, and log it
Irreversible, low-confidence decision	Pre-action human approval, showing the input and the flag reason
Compliance "proof"	Control + owner + evidence (BAA, config screenshot, log query)
Stakeholder says "seamless," "just a quick check"	Dig into the real constraint; ask all four discovery categories first
Tradeoff readout	Get, lose, reversal cost, compliance posture
Quality slowly declining, no alert	An owned trend trigger; a calendar audit in regulated settings
Outcome doc for a CFO	Auditable-control-backed before/after business metrics; never "reuse potential"
Standardized on AWS + in-region + latency sensitive	Bedrock as primary (explicit region) + Direct API as secondary for unregulated work
A shared Skill needs team-level rollback	Plugin (versioned); program caller → API Skill; repo asset → project Skill
Tests green, review approved, author can't explain the code	Pause the merge; checklist (understanding, security)
Quality declining gradually, no code change	Rerun eval; check model, prompt, retrieval drift
Model version unpinned, never re-validated added	Pin the version; gate every swap with a tiered test set and a pre-set threshold
A numbered-section policy corpus like the FMR added	Hierarchical chunking, section path kept on each chunk
Someone proposes a "never allowed" rule go into CLAUDE.md added	Governing layer instead: hook, permission, sandbox
A feasibility conclusion hinges on one	Run a technical spike first; don't conclude yet

unknown added	

Per-item method

1. Underline the specific risk or constraint the stem names.
2. Locate the matching step on the decision chain (overview page §3).
3. Eliminate options that solve a different risk, that substitute instruction for structure, or that only detect without preventing.
4. Pick the lightest option that removes the root cause.

← Module 5

Exam-Day Strategy →

SECTION 9 EQUIVALENT · DUE-DILIGENCE INDEX

Section 6 Objective Coverage Index

All 38 Exam Guide §6 objectives, mapped to where this package covers them. Objective names kept in English for exam alignment.

Domain · Objective	Covered at
D1 Translate business problems into Claude-based AI solutions	Overview §3; M1 §1.5; M2 §2.3
D1 Design end-to-end architectures (input → processing → output → feedback loops)	M1 §1.8, §1.13; M4 §4.3
D1 Select appropriate architectural patterns (workflow, agentic, augmented LLM)	M1 §1.4, §1.6
D1 Design multi-agent systems and orchestration strategies	M1 §1.7; M2 §2.2
D1 Apply decomposition techniques for complex problem solving	M1 §1.5
D1 Align solutions to business value pillars	M2 §2.3 (five pillars, ROI)
D2 Select appropriate Claude models based on trade-offs	M1 §1.10
D2 Design system prompts, templates, and guardrails	M1 §1.11
D2 Apply prompt engineering techniques (zero-shot, few-shot, chain-of-thought)	M1 §1.11
D2 Optimize context windows and manage token usage	M1 §1.2, §1.10; M2 §2.2
D2 Implement prompt reuse strategies (caching, modular prompts, Skills)	M1 §1.11; M5 §5.1
D3 Evaluate tool/agent configuration for capability bloat	M2 §2.4; Overview rule 1
D3 Analyze authentication and authorization requirements to identify security gaps	M2 §2.4; M3 §3.3
D3 Evaluate accuracy-latency trade-offs and justify configuration decisions	M2 §2.2; M4 §4.5
D3 Analyze observability challenges and select monitoring strategies at scale	M2 §2.4, §2.5
D3 Design a RAG pipeline with appropriate chunking and indexing strategies	M1 §1.9
D3 Apply retrieval strategies matched to data shape and query pattern	M1 §1.8, §1.9
D3 Evaluate connection protocols (MCP, API/CLI, agent-to-agent)	M1 §1.4, §1.12
D3 Evaluate progressive discovery vs. monolithic context strategy	M1 §1.10

D4	Define evaluation metrics (accuracy, latency, cost, safety, security)	M2 §2.1, §2.2
D4	Design evaluation datasets and test frameworks using mixed methodologies	M2 §2.1
D4	Conduct A/B testing and iterative improvements	M2 §2.5
D4	Diagnose system issues (prompt failure, hallucinations, model mismatch)	M2 §2.5; M5 §5.3
D4	Optimize token usage, latency, and cost-performance trade-offs	M1 §1.10, §1.11; M2 §2.2, §2.3
D4	Monitor system performance using logging and observability tools	M2 §2.4, §2.5; M4 §4.3
D5	Implement guardrails and safety controls	M3 §3.1, §3.3; Overview rules 1, 9, 10
D5	Identify risks, limitations, and failure modes of LLM systems	Overview §5; M2 §2.2; M3 §3.2
D5	Apply human-in-the-loop validation strategies	M3 §3.5; M1 §1.5
D5	Ensure compliance with regulations (GDPR, HIPAA, FedRAMP)	M1 §1.12; M2 §2.4; M3 §3.6
D5	Address ethical AI considerations (bias, fairness, transparency)	M3 §3.4
D6	Conduct structured discovery and requirement gathering	M4 §4.1
D6	Communicate architectural decisions and trade-offs	M4 §4.2; M1 §1.12
D6	Manage stakeholder feedback loops and expectation alignment (SLAs)	M4 §4.3
D6	Document architectures and provide implementation guidance	M4 §4.4
D6	Support lifecycle phases (discovery, design, handoff, monitoring, iteration)	M4 §4.5, §4.6
D7	Configure Claude tools and environments for teams (e.g. Claude Code)	M5 §5.1; M1 §1.12
D7	Improve developer workflows using AI-assisted tooling	M5 §5.2
D7	Support debugging and operational issue resolution	M5 §5.3

中文 对照表：38 个考点逐一映射到本资料包的具体位置，可直接跳转到薄弱考点。

← Exam Day

Official Sample Questions →

SECTION 8 OF THE EXAM GUIDE · YOUR CALIBRATION BASELINE — NEVER REUSED AS QUIZ ITEMS

Official Sample Questions

These three items are from Anthropic's Exam Guide v1.0 itself. They are not quiz practice — they are the baseline style and difficulty your tutor matches everything else against.

Sample 1 · Domain 3 — Integration

A team exposes a customer-support agent that can read tickets, draft replies, issue refunds, and delete user accounts. Support staff only ever need to read tickets and draft replies. Applying least-privilege principles, which change best reduces risk?

A. Add logging to the refund and delete tools so misuse can be audited later.

B. Remove the refund and delete tools from the agent's configuration entirely.

C. Keep all tools but add a confirmation prompt before refunds and deletions.

D. Replace the agent with a larger model that follows instructions more reliably.

Reveal answer

Sample 2 · Domain 2 — Models, Prompting & Context

An application sends the same 8,000-token system prompt and policy document on every request, followed by a short, varying user message. Latency and cost are both concerns. Which optimization most directly addresses both?

A. Truncate the policy document to the first 1,000 tokens.

B. Switch to the smallest available model regardless of task fit.

C. Place the static system prompt and policy before the dynamic content and enable prompt caching.

D. Move the policy document into a few-shot example block.

Reveal answer

Sample 3 · Domain 4 — Evaluation & Optimization

A RAG system suddenly returns confident but incorrect answers after a document refresh, while latency and model version are unchanged. What is the most likely first place to investigate?

A. The model weights have silently changed.

B. The retrieval/indexing step is returning irrelevant or stale chunks.

C. The temperature setting is too low.

D. The context window has shrunk.

Reveal answer

These three items are never reused as quiz content in any session — they are the baseline the tutor calibrates every generated item against. Real exam content is under NDA; everything else in this package is a practice item only, never a claim of having come from the live bank.

← Objective Index

Exam-Day Strategy →

SECTION 8 EQUIVALENT

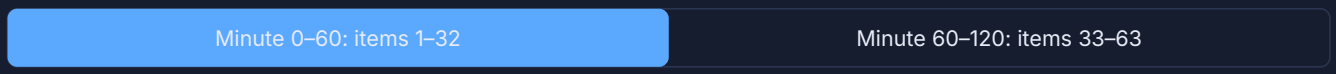
Exam-Day Strategy

Wed Dec 2, 2026 · 10:00am · online proctored, English · CCAR-P · Exam Guide v1.0

Format and scoring per Guide v1.0

- 63 items, 120 minutes, single-select and multiple-response.
- Scaled score 100–1,000; pass = 720.
- Criterion-referenced: measured against a fixed standard, not against other candidates.
- Score report: pass/fail, scaled score, and percent-correct by domain. **Domain percentages do not decide pass/fail** — only the total scaled score does.

Pacing



~1.9 min / item average
Stuck past 2.5 min → flag it and move on

63 items / 120 minutes. Target: roughly item 32 at the 60-minute mark.

Multiple-response items

The stem states how many to select. Select exactly that many; each selected option must stand on its own as correct — don't pad to reach a round number.

Per-item method

1. Underline the specific risk or constraint the stem names.
2. Locate the step on the decision chain (Overview §3).
3. Eliminate options that solve a different risk, substitute instruction for structure, or only detect without

preventing.
4. Pick the lightest option that removes the root cause.

Content character

The Professional exam reads like a client engagement, not a syntax quiz. It tests judgment, not API parameter names, CLI flags, or file paths. If an option hinges on syntax, judge it by the underlying concept instead.

中文 节奏：每题约 1.9 分钟，60 分钟时到第 32 题左右，卡住超过 2.5 分钟就标记跳过。

Watch your own tendencies

- Leaning toward detective controls (trace IDs, logging) over preventive ones.
- Substituting agency/government practice for the course's framing.
- Treating "add an instruction" as a structural fix.
- Answering correctly but not being able to state why — if you can't name the rule, you got lucky, not right.

Logistics

- Check-in opens 2:30pm (if testing at that slot) — confirm your own appointment's check-in window with Pearson VUE.
- Bring a valid, unexpired, government-issued photo ID matching your registration name exactly.
- Clear your desk; no phone, no smartwatch; stay in webcam view the whole session if testing online.
- Accept the confidentiality/NDA before the exam starts, or the session ends with no refund.

中文 带与报名姓名一致的有效证件，清空桌面，始终在摄像头范围内。

Readiness gate (your own rule)

Only declare yourself ready after two mixed 63-item mocks at 80%+ overall, with no domain below 65%. If Mock 2 on Mon Nov 30 misses that bar, reschedule before the 24-hour cutoff to keep the $175 fee.

← Cheat Sheet

Objective Index →